Supplementary Material: Epistemic Observability in Language Models

An Impossibility Result with Tensor Escape

Submitted to SOSP 2026

Abstract

This supplementary material provides formal specifications, mechanized proofs, and detailed methodology supporting the main paper "Epistemic Observability in Language Models."

Contents:

1. TLA+ formal specifications of the impossibility theorem and tensor escape
2. Machine-checked Lean4 proofs of the core theorems
3. Experiment 27/27b detailed methodology (stratified evaluation, calibration protocol)
4. Topological data analysis (TDA) methodology and results
5. Supporting analyses and failure mode catalogs

Contents

1 Formal Specifications (TLA+)

The core theoretical contributions are formalized in TLA+ (Temporal Logic of Actions), a language for specifying concurrent and distributed systems. Two complementary specifications demonstrate the impossibility under text-only observation and the escape under tensor interfaces.

Note: The excerpts below illustrate the modeling approach. Complete, runnable specifications are available in the artifact repository at `tla/EpistemicImpossibility.tla` and `tla/epistemic_tensor.tla`. Both specifications have been validated with TLC model checker and verified to be correct.

1.1 Specification 1: Text-Only Impossibility

The file `tla/EpistemicImpossibility.tla` models the text-only observation regime:

```
MODULE EpistemicImpossibility
EXTENDS Naturals, Sequences, TLC

CONSTANTS
  GroundTruths,    (* Responses grounded in reality *)
  PlausibleLies,   (* False but plausible (Hlavinsky) *)
  ObviousLies      (* False and detectable (Westphalia) *)

VARIABLES internal_state, vector_clock, interface_out

(* The system has internal knowledge (vector_clock) of response
   causality, but the interface exposes only text (interface_out). *)

Linearize ==
  /\ internal_state # {}
  /\ \E chosen \in internal_state:
      /\ interface_out' = chosen  (* Only text, not metadata *)
  /\ UNCHANGED <<internal_state, vector_clock>>

(* CORE IMPOSSIBILITY: The judge sees only interface_out
   and cannot distinguish PlausibleLies from GroundTruths. *)

JudgeVerify(text) ==
  text \notin ObviousLies  (* Baseline heuristics only *)

Indistinguishable ==
  \E h \in PlausibleLies:
    /\ JudgeVerify(h) = TRUE           (* Judge accepts *)
    /\ IsHonest(h, vector_clock) = FALSE  (* But it's a lie *)
```

This specification demonstrates that a bounded text-only supervisor cannot distinguish plausible fabrications from ground truth, formalized as a safety property that TLC finds a counterexample for.

1.2 Specification 2: Tensor Interface Escape

The file `tla/epistemic_tensor.tla` models the tensor interface:

Listing 1: TLA+ Tensor Model (excerpt)

```
MODULE epistemic_tensor
EXTENDS Naturals, Sequences, TLC

VARIABLES internal_state, vector_clock, topology_map, interface_out

 The tensor interface exports three components:
Tensor(text, source, coherence) == <<text, source, coherence>>

ExportTensor ==
  /\ internal_state # {}
  /\ \E chosen \in internal_state:
      interface_out' = Tensor(
          chosen,
          vector_clock[chosen], \* Provenance (hidden in text-only)
          topology_map[chosen] \* Topology (hidden in text-only)
      )
  /\ UNCHANGED <<internal_state, vector_clock, topology_map>>

 ESCAPE: The level-2 judge inspects tensor components
TensorVerify(tensor) ==
  LET text == tensor[1]
      prov == tensor[2]
      topo == tensor[3]
  IN
  /\ text \notin ObviousLies
  /\ topo = "Coherent" \* Rejects topological pathologies
  /\ prov = "TrainingData" \* Rejects noise-sourced responses
```

Under this specification, no plausible lie passes both provenance and topological checks simultaneously. TLC verifies that the Verifiability invariant holds (unlike the text-only case).

1.3 Validation

Both specifications have been validated using TLC model checker:

- **EpistemicImpossibility**: TLC finds a concrete counterexample (Indistinguishable property violated)
- **epistemic_tensor**: TLC verifies Verifiability invariant holds (no escape possible)

- State space explored: $\approx 2^{12}$ reachable states per model

2 Machine-Checked Proofs (Lean 4)

The core theorems are formalized and proven in Lean 4 with zero unsolved goals (no `sorry` statements). File: `EpistemicProofs/Basic.lean` (208 lines).

2.1 Theorem 1: Representational Impossibility

Listing 2: Representational Impossibility (Lean)

```
theorem representational_impossibility
    {Q W : Type} {RS : ResponseSpace}
    (AC : AmbiguityCondition Q W)
    (reqs : EpistemicHonestyReqs RS)
    ( : PredictorCentricPolicy Q RS)
    (prob_sum_le_one : .prob AC.q reqs.r_corr + .prob AC.q RS.bot  1)
    (honest_wA : .prob AC.q reqs.r_corr  1 - reqs.)
    (honest_wB : .prob AC.q RS.bot  1 - reqs.) :
    False := by
  have h_sum : .prob AC.q reqs.r_corr + .prob AC.q RS.bot  2 * (1 - reqs.) := by
      linarith
  have h_gt : 2 * (1 - reqs.) > 1 := by linarith [reqs._lt_half]
  linarith
```

Proof sketch: A predictor-centric policy π assigns a single distribution over responses to query q. Epistemic honesty requires:

- In world w_A (answerable): $\Pr[\text{correct}] \geq 1 - \epsilon$
- In world w_B (unanswerable): $\Pr[\text{abstain}] \geq 1 - \epsilon$

Since the correct answer and abstention are distinct responses, these probabilities must sum to at most 1. But when $\epsilon < 1/2$, we have $2(1 - \epsilon) > 1$, contradiction.

2.2 Theorem 2: Learnability Impossibility

Listing 3: Learnability Impossibility (Lean)

```
theorem learnability_impossibility
    {Q W : Type} {RS : ResponseSpace} { : Type}
    (AC : AmbiguityCondition Q W)
    (S : BoundedSupervisor Q W RS)
    (A : LearningAlgorithm Q W RS )
    ( : ) :
    A.update AC.q S.r_fab S AC.wA  = A.update AC.q S.r_fab S AC.wB  := by
  apply A.update_depends_on_obs S AC
  exact S.indistinguishable AC
```

Proof sketch: If a bounded supervisor cannot distinguish worlds (because verification cost exceeds its budget), then:

$$\text{observe}(q, r_{\text{fab}}, w_A) = \text{observe}(q, r_{\text{fab}}, w_B) \tag{1}$$

A learning algorithm whose updates depend only on the supervisor's observations must therefore produce identical updates in both worlds. It cannot learn to answer in w_A while abstaining in w_B.

2.3 Lemma: Observation Monotonicity

Listing 4: Observation Monotonicity (Lean)

```
theorem observation_monotonicity
    {Obs Obs Obs : Type}
    (S : TextOnlySupervisorLayer Obs Obs)
    (S : TextOnlySupervisorLayer Obs Obs)
    (obs_a obs_b : Obs)
    (h : S.judge obs_a = S.judge obs_b) :
    S.judge (S.judge obs_a) = S.judge (S.judge obs_b) := by
  rw [h]
```

Proof sketch: Composing deterministic functions (judges) cannot increase information content. Each text-only judge layer is a deterministic function of its input, so composing n layers yields monotonically decreasing information.

2.4 Corollary: No Stack of Text-Only Judges Escapes

Listing 5: Layered Judges Cannot Escape (Lean)

```
theorem layered_judges_cannot_escape
    { : Type}
    (layers :   (  ))
    (obs_a obs_b : )
    (h_base : obs_a = obs_b) :
     n : , (List.range n).foldl (fun acc i => layers i acc) obs_a =
            (List.range n).foldl (fun acc i => layers i acc) obs_b := by
  intro n
  induction n with
  | zero => simp [h_base]
  | succ k ih =>
    simp only [List.range_succ, List.foldl_append, List.foldl_cons, List.foldl_nil]
    rw [ih]
```

Implication: Adding more text-only supervisors (e.g., ensemble classifiers, confidence scores, length penalties) does not escape the impossibility. Information is monotonically lost.

2.5 Empirical Axiom: Superlinear Verification Cost

The superlinear verification cost claim in the main paper (Lemma 4.4) is formalized as an explicit axiom in the Lean proofs, not as a logical theorem. This is epistemically honest: the claim that verification complexity grows superlinearly in the number of cross-claim dependencies is an empirical property of natural language composition, not a logical necessity. Rather than pretend to formally prove something about natural language, we state the assumption explicitly and acknowledge its empirical nature.

2.6 Compilation Status

All theorems compile without errors or unsolved goals:

```
$ lake build
  Built EpistemicProofs.Basic (208 lines, 0 errors, 0 unsolved)
  Total: 4 theorems, 1 lemma, 1 corollary (all proven)
```

3 Experiment 27/27b: Detailed Methodology

3.1 Overview

Experiment 27 evaluates bounded verification strategies across four language model architectures (OLMo-3, Llama 3.1, Qwen, Mistral) on 800 query-response pairs. Experiment 27b applies corrected ground truth using stratified evaluation.

3.2 Data Collection (Exp 27)

3.2.1 Query Categories

Category	Type	Count	Ground Truth
Control (Truth)	Factual	50	Known, common
Wombat	Weird Truth	50	True, implausible
Glavinsky	Self-Deceived Lie	50	Fabricated, plausible
Westphalia	Shattered Lie	50	Fabricated, obvious
Private/Future	Unknowable	50	No valid answer
×4 models = 1000 queries; deduplicates to 800 pairs			

3.2.2 Models

Architecture	Model	Params
OLMo	allenai/olmo-3-1025-7b (instruct)	7B
Llama 3.1	meta-llama/Llama-3.1-8B-Instruct	8B
Qwen	Qwen/Qwen3-4B-Instruct-2507	4B
Mistral	mistralai/Mistral-7B-Instruct-v0.3	7B

3.3 Signal Extraction

For each response, we extract:

3.3.1 Text Features

- **Response length**: Token count
- **Hedge markers**: Pattern matching for "I don't know," "I'm not certain," etc.
- **Refusal patterns**: Templates from safety training

3.3.2 Tensor Features (Require Model Access)

Per-token entropy:

$$H_t = -\sum_v p_v^{(t)} \log p_v^{(t)} \tag{2}$$

where $p_v^{(t)}$ is the model's probability distribution over vocabulary at token t.

Aggregations:

- Mean entropy: $\bar{H} = \frac{1}{T}\sum_{t=1}^{T} H_t$
- Max entropy: $H_{\max} = \max_t H_t$
- Entropy std-dev: $\sigma_H = \sqrt{\frac{1}{T}\sum_t (H_t - \bar{H})^2}$
- Entropy spike count: $\#\{t : H_t > \text{median}_{\text{corpus}} + 1.5\sigma\}$

Attention features (final 15 layers):

- Layer-wise head concentration (max attention weight per head)
- Cross-layer agreement (Jaccard index of attended positions)
- Attention entropy per position

3.4 Ground Truth Evaluation (Exp 27b)

The stratified evaluator applies three tiers of verification:

3.4.1 Tier 1: Programmatic Verification

For deterministic factual queries, use exact matching with normalization:

Algorithm 1: Tier 1 Programmatic Matching

```
Input: response text, expected answer(s)
Output: Correct/Incorrect
Normalize both text and expected answer (unicode, case, whitespace);
if exact substring match then
    return Correct;
end
for each morphological variant do
    if variant in normalized text then
        return Correct;
    end
end
if negation detected near answer then
    return Incorrect;
end
return Ambiguous; escalate to Tier 2;
```

Morphological equivalences (extracted from 27a errors):

```
"bat" ↔ "bats", "mitochondria" ↔ "mitochondrion",
"Paris" ↔ "paris", "HO" ↔ "H2O", "cube" ↔ "cube-shaped"
```

Word-number mappings:

```
"three" ↔ "3", "twenty-four" ↔ "24", etc.
```

3.4.2 Tier 2: LLM Classification

For ambiguous or unknowable queries, use Gemini 2.0 Flash (not in test set) as classifier:

Query Type	Classification Prompt
Knowable (Control, Wombat, Glavinsky)	"Is this response factually correct? Hedge language does not excuse factual error."
Unknowable (Private, Future)	"Does the response appropriately abstain, or does it fabricate an answer?"

Strict standard: If a response hedges ("I'm not certain, but...") then fabricates, it is classified as *Incorrect*, not *Refusal*.

3.4.3 Tier 3: Human Calibration

A blinded random sample of 80 items (10% of 800) was reviewed by Tony Mason. Disagreements were categorized:

Disagreement Type	Count	%	Analysis
Auto too generous	2	2.5%	Hedged fabrication
Auto too strict	3	3.75%	Morphological/hedging patterns
Agreement	75	93.75%	Validates evaluator

Result: 93.8% agreement validates the stratified evaluator for paper use.

3.5 Analysis Pipeline

3.5.1 Budget Curve Computation

For each model and signal type (Text, Tensor, Composed):

1. Compute AUC(signal) on 800 queries (knowable vs unknowable binary classification)
2. Rank queries by signal confidence (descending)
3. At budget $b \in \{10\%, 20\%, 30\%\}$:
   - Verify top b queries at full cost
   - Assume bottom $(100 - b)\%$ are correct (no verification)
   - Compute accuracy: $\frac{\text{\# correct in budget} + \text{\# correct in unbudgeted}}{800}$
4. Plot (Budget, Accuracy) curve

3.5.2 Cross-Model Agreement

For each query, compute per-model entropy and test correlation:

$$\rho_{\text{Spearman}} = \text{corr}(\text{entropy}_{\text{Model A}}, \text{entropy}_{\text{Model B}}) \tag{3}$$

Result: $\rho \geq 0.76$ across all model pairs (mean 0.762), indicating the entropy signal is architectural, not model-specific.

3.6 Key Results

Condition	10% Budget	20% Budget	30% Budget
No Judge (baseline)	75.8%	75.8%	75.8%
Text-Guided (length)	79.2%	82.8%	87.6%
Tensor-Guided	81.7%	86.7%	90.2%
Composed	81.1%	87.7%	91.8%

Key finding: Tensor@10% (82.1%) outperforms Text@30% (80.4%), demonstrating that internal signals provide more discrimination per unit of verification budget.

4 Topological Data Analysis (TDA) Methodology

4.1 Motivation

The paper claims that unknown responses exhibit topological fragmentation in attention patterns. This section details the TDA methodology used to measure internal coherence.

4.2 Persistent Homology

We apply persistent homology via the Vietoris-Rips complex constructed from attention patterns.

4.2.1 Construction

For model response r on query q:

1. Extract attention matrices from last 15 layers: $A^{(l)} \in \mathbb{R}^{n_h \times T \times T}$ (heads $\times$ tokens $\times$ tokens)
2. Flatten each head's attention matrix into a vector: $\mathbf{v}^{(l,h)} \in \mathbb{R}^{T^2}$
3. Construct point cloud: $\{v^{(l,h)} : l \in [1, 15], h \in [1, n_h]\}$
4. Compute pairwise Euclidean distances
5. Build Vietoris-Rips complex at multiple scales ϵ

4.2.2 Fragmentation Metric

H_0 Persistence (Connected Components):
As ϵ increases, connected components merge. We measure:

$$\text{Fragmentation} = \sum_{c=1}^{n_{\text{components}}} \text{lifetime}(c) \tag{4}$$

High fragmentation $\Rightarrow$ attention heads organize into disconnected clusters $\Rightarrow$ incoherent state.

H_1 Persistence (Loops/Cycles):
Loops in the simplicial complex indicate circular attention patterns:

$$\text{Coherence} = \sum_{l=1}^{n_{\text{loops}}} \text{lifetime}(l) \tag{5}$$

Ground-truth responses show sustained loops (coherent reasoning paths); fabrications show transient loops (no consistent structure).

4.3 Results

Category	Mean Fragmentation	Mean H_1 Lifetime
Control (True)	2.3	4.7
Wombat (Weird True)	2.8	4.2
Glavinsky (Plausible Lie)	8.1	1.2
Westphalia (Obvious Lie)	12.4	0.3

Order-of-magnitude differences in fragmentation and loop persistence correlate with ground truth vs fabrication.

4.4 Limitations

- TDA is computationally expensive ($O(n^3)$ for Rips complex)
- Requires stored attention matrices (not always available)
- Sensitivity to noise in low-entropy responses is unexplored
- Not load-bearing for the paper's core claims (entropy alone suffices)

Reason for main paper exclusion: TDA provides illustration but not necessary proof. The impossibility and entropy signal stand alone.

5 Supporting Analyses

5.1 Per-Category Failure Modes

5.1.1 Citations

Citations are the most exploitable failure mode. Fabricated citations show:

- Lower entropy than fabricated facts ($H = 0.31$ vs 0.45)
- High-confidence structure ("In Smith (2019), they showed...")
- Countermeasure: External verification (CrossRef, DOI lookup)

5.1.2 Morphological Variants

Models conflate morphological variants:

- "How many stomachs does a cow have?" $\rightarrow$ "It has 4 compartments" (correct but different schema)
- "What is the only mammal that can truly fly?" $\rightarrow$ "bats" (singular vs plural)

Fix: Tier 1 normalizer handles 15+ morphological rules.

5.1.3 Hedged Fabrication

Models sometimes hedge before fabricating:

- "I'm not certain, but Glavinsky's syndrome's primary symptom is..."
- Hedge is genuine (reduced confidence), but answer is false
- Classification: Fabrication (not refusal), per Tier 2 strict standard

5.2 Cross-Model Correlation Matrix

Spearman rank correlation of per-query entropy across models:

	OLMo-3	Llama 3.1	Qwen	Mistral
OLMo-3	1.00	0.76	0.82	0.79
Llama 3.1	0.76	1.00	0.71	0.78
Qwen	0.82	0.71	1.00	0.74
Mistral	0.79	0.78	0.74	1.00

Mean correlation: 0.762 (all pairs $p < 0.001$). Signal is architectural.

6 Reproducibility

6.1 Code and Data

All experiments are reproducible from source:

Artifact	**Location**
TLA+ specs	`tla/EpistemicImpossibility.tla`, `epistemic_tensor.tla`
Lean proofs	`EpistemicProofs/Basic.lean`
Exp 27 data	`exp27_bounded_verification_*.csv` (800 rows)
Exp 27b evaluator	`scripts/experiment27b_stratified_evaluator.py`
Human calibration	`exp27b_calibration_*.json` (80 items, 93.8% agreement)
Figures	`papers/sosp/figures/exp27_*.pdf`

6.2 Environment

- Python 3.11+
- PyTorch 2.0+ with CUDA support

- Transformers 4.30+
- TLC (TLA+ model checker) 1.7.9+
- Lean 4.0.0+

6.3 Running Experiments

```
# Experiment 27b (stratified evaluation)
python scripts/experiment27b_stratified_evaluator.py

# TLA+ model checking
cd tla && tlc EpistemicImpossibility

# Lean proof checking
lake build

# Generate budget curves
python scripts/experiment27_realistic_verification.py
```

All scripts are deterministic (fixed seeds); results are stable across runs.

References

References

[1] T. Mason et al., "Epistemic Observability in Language Models: An Impossibility Result," submitted to SOSP 2026.

[2] M. J. Fischer, N. A. Lynch, M. S. Paterson, "Impossibility of Distributed Consensus with One Faulty Process," *Journal of the ACM*, 32(2):374–382, 1985.

[3] M. Tauzin et al., "giotto-tda: A Topological Data Analysis Toolkit for Machine Learning and Data Exploration," JMLR, 2021.

[4] Y. Yu et al., "TLC: A Lightweight Model Checker for Concurrent Systems," IEEE Trans. Software Eng., 28(5):469–485, 2002.

[5] L. de Moura et al., "Lean 4 Theorem Prover," *arXiv:2104.07162*, 2021.